

MongoDB Collection Design Justification

Task 1.1 Collection Design

Vehicle Collection

The vehicles collection stores relatively static vehicle registration and ownership information loaded from vehicle.csv. This collection acts as master reference data for the streaming application.

Each vehicle document will store:

```
{
  "car_plate": String,
  "owner_name": String,
  "owner_addr": String,
  "vehicle_type": String,
  "registration_date": datetime,
  "created_at": datetime,
  "updated_at": datetime | null
}
```

This collection itself is required since we need to store the vehicle ownership and registration data, which is static and updated infrequently compared to the traffic events from our cameras which require frequent updates.

If we were to embed all the vehicle data into every violation document, there would be a high number of duplication because the same owner information can be repeatedly stored across many violations. To combat this, we will only store the car_plate inside our violation collection so that we can simply conduct a lookup in our vehicle collection when we need information regarding it.

So, the main idea behind separating this vehicle collection is to:

1. reduce redundancy
2. improve consistency
3. enabling independent updates

For example, if we find that a vehicle is acquired by a new owner or requires a registration update, we simply need to update this information in the vehicles collection and not in the violations collection, which reduces the number of operations we need to do.

To further improve this collection, we also use indexing on the car_plate attribute, namely:

```
("car_plate", 1)
unique=True
```

We select this attribute since `car_plate` acts as a natural unique identifier for vehicles. Furthermore, an ascending ordering was also placed on this attribute since it acts as the primary lookup attribute and natural identifier for each vehicle.

Ascending ordering was selected (although descending will perform the same) because vehicle lookups are typically equality-based rather than order-sensitive. During streaming processing, vehicle records are retrieved using exact matching:

```
db.vehicles.find_one({
  "car_plate": plate
})
```

Camera Collection

The camera collection stores static data regarding the roadside monitoring cameras that provides us the data stream. The document itself will store:

```
{
  "camera_id": int,
  "created_at": datetime
  "location": {
    "type": "Point",
    "coordinates": [double, double] ([long, lat])
  },
  "position": double, (used to calculate distance between each camera)
  "speed_limit": int
}
```

This collection is required since similarly to vehicle, updates will be infrequent to the actual cameras. Furthermore, the information for each camera, mainly the position and speed_limit will be frequently used for processing violations. Having this collection avoids us repeatedly embedding location and speed-limit into every event or violation, reducing duplicates.

Furthermore, the location of the camera is stored using the GeoJSON standards by MongoDB, which is why we use "type": "Point".

To further improve this collection, an indexing strategy was also utilized, namely:

```
( "camera_id", 1)
unique=True
```

This is done to ensure uniqueness, fast camera lookup (if our system scales), and efficient information retrieval.

Violation Collection

Violations are stored in a separate collection as this collection will handle outputs from our Spark streaming application. Unlike the camera and vehicle collection, this collection will receive a lot of writes during runtime.

The main point of this separation is important because:

1. We can prevent excessive writing to static collections Violation events may arrive continuously in real time. If violations were embedded directly into the vehicles collection, that means we would be updating the vehicle document for every detected violation.
2. Scalability improvement A single vehicle may accumulate many violations over time.

Embedding all violations inside a vehicle document could lead to:

- oversized documents
- slower updates
- inefficient document rewrites

MongoDB documents have a size limit (~16 MB), so unbounded growth is undesirable.

By separating violations, streaming writes remain manageable and scalable.

3. Reduces Contention During Streaming

The Spark streaming application continuously inserts or updates violations.

Keeping violations separate minimizes write contention on master data (vehicles, cameras) and reduces the risk of accidentally modifying reference information during streaming ingestion.

This is especially important in real-time systems where:

- ingestion speed matters
- retries may occur
- multiple updates happen concurrently

The document itself will be grouped by the car_plate and date of violation, to satisfy the requirement of:

multiple violations occurring on the same day should be merged into a single daily violation record.

So, the document structure will be as such:

```
{
  "car_plate": String,
  "date": datetime,
  "violations": [
    {
      "type": "instant", (if we detect on a single camera that speed > limit)
      "camera_id": int,
      "speed": double
    },
    {
```

```
    "type": "average", (if the time traveled between 2 cameras is too fast)
    "start_camera": int,
    "end_camera": int,
    "avg_speed": double
  }
]
```

Violations are embedded within a single daily document rather than stored as separate documents, mainly because:

1. violations for a vehicle/day are commonly accessed together
2. avoids expensive joins
3. reduces query complexity
4. aligns with MongoDB document-oriented design

This can also help in supporting efficient streaming as we can stream updates through the usage of `$push`.

An indexing strategy was also applied to this collection, namely:

```
("car_plate", 1),
("date", -1)
```

This index optimizes:

- vehicle-specific lookups
- daily violation merging
- recent-history queries

The descending order on date (-1) prioritizes retrieval of newer records.

Embedding vs Referencing Embedding improves read efficiency and simplifies retrieval but may increase document size over time.

However, since violations are grouped daily rather than indefinitely, document growth remains manageable.

If we were to use referencing, it will have a higher query cost since the assignment itself wants us to merge same-day violations into a single document, but if we were to use referencing then we need to retrieve and merge multiple records. Furthermore, this will have higher aggregation costs as we will need to merge these records. Finally, it will also have worse stream write efficiency as with our embedding approach, we can simply use `$push`. However with referencing, we will have to insert new records everytime a violation is found, which will then result in the two drawbacks previously mentioned.

Task 1.2 Collection Relationship

`vehicles` and `cameras` are stand-alone collections that do not reference any other collections

`violations` reference `vehicles` via the attribute `car_plate` and also `cameras` via the attribute `camera_id` instead of embedding the full documents.

This is mainly done to reduce duplicate information and improve consistency since if we were to update vehicle or camera information, violations will stay "up-to-date" with this change as it is simply referencing the unique key of the collections.

The `violations` collection itself also uses embedding over referencing to better optimize for:

1. High write throughput
2. Efficient reads
3. Minimal join cost
4. Streaming correctness